

Client Selection in Federated Learning under Non-IID Data and Concept Drift

Machine Learning Project Report (CS-503/AI-211)

Kammari Harshavardhan Kothi Vishwan Puchakayala Rohith

Abstract

Federated learning enables collaborative model training without sharing raw client data, but the choice of participating clients strongly affects convergence, accuracy, and communication cost. In this project, we study client selection on MNIST under non-IID data and compare a greedy diversity-based strategy inspired by DivFL with a Shapley-value-based strategy inspired by S-FedAvg, where Shapley values are approximated via Monte Carlo sampling over random client permutations. We evaluate both methods across CNN, LeNet, Logistic Regression, SVM, and Random Forest, measuring round-wise test accuracy, convergence speed, and cost over 100 federated rounds. Later tasks extend the framework to concept drift and an adaptive drift-aware client selection method (SW-ShUCB) that combines sliding-window Shapley rewards with UCB exploration.

Keywords: Federated Learning, Client Selection, MNIST, Non-IID Data, DivFL, Shapley Value, Concept Drift, SW-ShUCB

1 Introduction

Federated learning (FL) trains a global model across distributed clients while keeping local data private. In a standard FL round, the server broadcasts the current model to a subset of clients, each client performs local training, and the server aggregates the returned updates. The classic FedAvg objective is

$$F(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w), \quad (1)$$

where n_k is the number of samples on client k and $n = \sum_k n_k$. In practice, client data are often non-IID and unbalanced, so selecting the right subset of clients matters as much as the learning algorithm itself. This project studies that selection problem on MNIST across three tasks: Task 1 compares greedy diversity against Shapley-style scoring under static non-IID conditions; Task 2 evaluates both strategies under concept drift with clean and drifted test tracking; and Task 3 proposes an adaptive bandit-based selector (SW-ShUCB).

2 Background

2.1 Federated Learning and Client Participation

In each federated round, the server selects a subset of clients, sends them the current parameters, and aggregates their locally trained updates. Raw data never leave the clients, but the method is sensitive to client heterogeneity. With strongly label-skewed distributions (from shard-based partitioning), uniform random selection can bias gradient updates toward dominant classes and slow convergence on under-represented ones.

2.2 Greedy Diversity-Based Selection

The greedy strategy chooses clients whose distributions differ as much as possible from those already selected. The objective is $S^* = \arg \max_{|S|=K} f(S)$, where $f(S)$ increases with cohort diversity. Clients are added one at a time by maximising the marginal gain:

$$\Delta(j \mid S) = f(S \cup \{j\}) - f(S), \quad (2)$$

using cosine distance $d(j, s) = 1 - h_j \cdot h_s / (\|h_j\| \|h_s\|)$ between normalised label histograms—the operational idea behind DivFL [2]. This favours clients that contribute new class coverage rather than repeatedly sampling already well-represented digits.

2.3 Shapley-Style Client Scoring

The Shapley value measures each client’s marginal contribution:

$$\phi_i(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi(N)} [v(C_\pi(i) \cup \{i\}) - v(C_\pi(i))]. \quad (3)$$

Because exact computation is expensive, we use a Monte Carlo approximation [3]:

$$\hat{\phi}_i = \frac{1}{R} \sum_{r=1}^R [v(C_{\pi_r}(i) \cup \{i\}) - v(C_{\pi_r}(i))], \quad (4)$$

where π_r is the r -th randomly sampled permutation and the resulting proxy score ranks clients by estimated utility. This proxy is computed with a lightweight local model so the ranking remains practical inside each round.

Table 1: Task 1 Hyperparameters.

Hyperparameter	Value
Total clients (N)	50
Candidate pool per round	20
Clients per round (K)	10
Global rounds	100
Local epochs / batch size	3 / 32
Proxy val. set size	5,000
Shards per client	4
CNN optimizer	Adam, $\text{lr} = 10^{-3}$
Strategies	Greedy (cosine), Shapley-proxy (SGD)

3 Task 1: Evaluating Client Selection Strategies

3.1 Experimental Setup

We use MNIST with $N = 50$ clients. Training data are split into a 55,000-sample pool and a 5,000-sample stratified proxy validation set. Shard-based partitioning (4 shards per client, sorted by label) produces strongly label-skewed distributions where most clients hold samples from only 2–4 digit classes. Each round, 20 clients are sampled uniformly and $K = 10$ are selected. Models train for 3 epochs (batch size 32) with FedAvg over 100 rounds, evaluated on a clean 10,000-sample test set after each round.

Five model families are evaluated: Simple CNN (two Conv+MaxPool blocks, Dense 128, Dropout 0.3) and LeNet (C6-P-C16-P-FC120-FC84) in TensorFlow/Keras with Adam ($\text{lr} = 10^{-3}$); and Logistic Regression (lbfgs, 300 iter), SVM (SGDClassifier, hinge loss, `partial_fit`), and Random Forest (30 estimators) from scikit-learn. A `StandardScaler` fitted on the pool is applied to all flat-input models. Table 1 summarises the configuration.

3.2 Model Architectures

Figure 1 shows the LeNet reference architecture. The network takes a grayscale MNIST image of size $1 \times 28 \times 28$, applies Conv1 (6 filters, 5×5), MaxPool1 (2×2 , stride 2), Conv2 (16 filters, 5×5), MaxPool2, then flattens to 256 features before Dense1 (120 ReLU), Dense2 (84 ReLU), and a 10-way Softmax output. The Simple CNN baseline is shallower:

Conv2D(32)→MaxPool→Conv2D(64)→MaxPool
→Dense(128)→Dropout(0.3)→Dense(10)

so it has fewer parameters than LeNet while still capturing the main MNIST structure. Both models use Adam ($\text{lr} = 10^{-3}$) and respond differently to non-IID sampling, making them useful for comparing selection policies.

3.3 Results

3.3.1 Overall Performance

Both strategies achieve strong accuracy on MNIST, with CNN models converging within the first 20–30 rounds. Greedy uses cosine distance between label histograms, starting from a random seed client and iteratively adding the candidate that maximises minimum pairwise distance to the selected set. Shapley-proxy trains a small

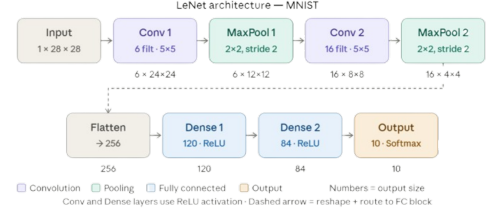


Figure 1: LeNet architecture for MNIST: Input → Conv1 → MaxPool1 → Conv2 → MaxPool2 → Flatten → Dense1 → Dense2 → Output.

SGDClassifier ($\alpha = 10^{-4}$, 100 iter) per candidate on 200 local samples and selects top- K by proxy validation accuracy. Figure 2 shows round-wise trajectories for all model families.

3.3.2 Final Performance Across Models

Table 2 reports final, best, and mean accuracy. Greedy consistently matches or outperforms Shapley-proxy; the gap is most pronounced for classical models. CNN and LeNet exceed 98.8% under both strategies, while Logistic Regression and SVM show larger gaps ($\sim 1.8\%$ and $\sim 2.0\%$) reflecting their limited representational capacity.

Table 2: Final Performance Metrics (100 rounds, $K = 10$).

Model	Strategy	Final	Best	Mean
Simple CNN	Greedy	0.9922	0.9936	0.9912
Simple CNN	Shapley-proxy	0.9910	0.9924	0.9897
LeNet	Greedy	0.9898	0.9920	0.9890
LeNet	Shapley-proxy	0.9886	0.9908	0.9869
Random Forest	Greedy	0.9398	0.9441	0.9357
Random Forest	Shapley-proxy	0.9250	0.9470	0.9224
SVM	Greedy	0.9133	0.9158	0.9119
SVM	Shapley-proxy	0.9113	0.9141	0.9084
Logistic Reg.	Greedy	0.8870	0.8948	0.8865
Logistic Reg.	Shapley-proxy	0.8686	0.8920	0.8775

3.3.3 Effect of K and Computation Cost

Tables 3–4 show final accuracy at $K \in \{5, 10, 20\}$. Accuracy increases sub-linearly: gains are sharpest from $K = 5$ to $K = 10$ and nearly saturate beyond, reflecting diversity saturation where additional clients from the shard-constrained pool contribute diminishing label coverage. Greedy runs $\mathcal{O}(N^2)$ cosine distance comparisons (< 1 ms per round for pool size 20). Shapley-proxy trains 20 SGDClassifiers per round—more expensive but far cheaper than true Monte Carlo Shapley ($2^{|S|}$ coalitions). Greedy is the preferred baseline: better accuracy, lower cost.

Table 3: Simple CNN final accuracy under varying K .

Strategy	$K = 5$	$K = 10$	$K = 20$
Greedy	0.9901	0.9922	0.9928
Shapley-proxy	0.9882	0.9910	0.9916

Table 4: Logistic Regression final accuracy under varying K .

Strategy	$K = 5$	$K = 10$	$K = 20$
Greedy	0.8812	0.8870	0.8901
Shapley-proxy	0.8634	0.8686	0.8741

4 Task 2: Performance under Concept Drift

4.1 Drift Simulation

Task 2 extends the 50-client setup with phase-based drift applied only to training data, while the test set remains clean. This separates two effects: whether the selector can survive changing client data, and whether the model can still generalise to the original MNIST distribution. The notebook tracks both **clean test accuracy** and **drifted test accuracy**. It also records the immediate post-drift drop, short-term recovery, and the clean-vs-drift gap.

Phase-4 Label Shift. Four label-remapping events fire at rounds $\{20, 40, 60, 80\}$. The remappings are cumulative: Phase 1 swaps $\{0 \leftrightarrow 1, 2 \leftrightarrow 7\}$; Phase 2 adds $\{3 \leftrightarrow 8, 4 \leftrightarrow 9\}$; Phase 3 adds $\{5 \leftrightarrow 6\}$. This corrupts class semantics directly and is therefore the harsher scenario.

Phase-4 Feature Shift. Four transformation events fire at the same rounds: progressive pixel shifts (up to 5 px via `np.roll`), small rotations (up to 15°), Gaussian blur ($\sigma \leq 1.0$), and pixel noise ($\sigma_{\text{noise}} \leq 0.05$). The transformation direction varies per client, making the feature drift more realistic than a global synthetic perturbation.

Table 5: Task 2 Concept Drift Parameters.

Parameter	Value
Scenarios	<code>phase4_label, phase4_feature</code>
Drift events	4 (at 20%, 40%, 60%, 80% of rounds)
Label corruption max	15% per event (cumulative)
Feature shift max	5 px, 15° rotation, $\sigma = 1.0$ blur
Evaluation set	Clean (no drift applied)
Analysis metrics	Final/best mean accuracy, drift gap, drop

4.2 Results & Discussion

Both strategies degrade at each drift event and then recover at different speeds. The key pattern is that label drift is consistently more damaging than feature drift, because it changes the meaning of the labels themselves rather than just input appearance. CNNs are more resilient than linear models in both scenarios, while Logistic Regression and SVM are the quickest to lose accuracy under label remapping. A useful way to read the curves is through three phases: pre-drift stability, immediate post-drift collapse, and recovery. The clean curves show whether the model can re-centre itself after the drift event, while the drifted curves reveal whether the learned representation remains useful under the shifted evaluation distribution.

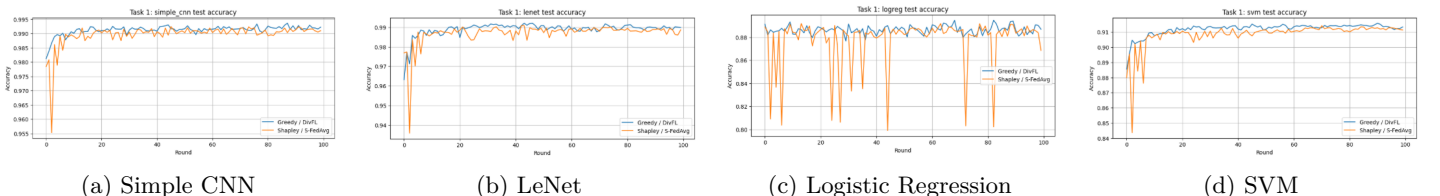


Figure 2: Round-wise test accuracy over 100 federated rounds under Greedy and Shapley-proxy selection.

5 Task 3: SW-ShUCB Adaptive Client Selection

5.1 Motivation and Design

5.1.1 Multi-Armed Bandit View

Task 3 reframes client selection as a multi-armed bandit problem. Each client is treated as an arm, and choosing a client corresponds to pulling that arm. After each round, the server observes a reward signal reflecting how helpful the selected client was for the global model. This is natural because the server repeatedly chooses a limited subset under a communication budget while balancing exploitation of useful clients and exploration of uncertain ones.

5.1.2 Why a Stationary Bandit Is Not Enough

A plain multi-armed bandit assumes stationary arm rewards. That assumption breaks under concept drift. In federated learning, client data can change suddenly: labels may be remapped, input features distorted, or client populations shifted. Rewards collected in earlier rounds then become stale and may no longer reflect current client usefulness. A stationary bandit can over-exploit clients that were good before the drift and under-explore clients that are now more informative.

5.1.3 Shapley Reward with a Sliding Window

Task 3 uses a Shapley-inspired reward and a sliding window. The reward is based on a leave-one-out (LOO) approximation to client utility:

$$\phi_k = \text{acc}(S_t) - \text{acc}(S_t \setminus \{k\}),$$

where $\text{acc}(S_t)$ is the current round accuracy using selected set S_t . Rather than averaging rewards over the entire history, we store only the most recent W rewards for each client in a deque. The sliding window acts as a forgetting mechanism: older rewards are removed automatically, so the client score tracks the current regime instead of the obsolete past.

5.1.4 UCB Exploration for Client Diversity

Even if a client has looked weak recently, it may become useful after drift. The UCB bonus ensures that clients with fewer recent selections still get explored, preventing starvation. The final score combines the short-term Shapley signal with exploration pressure:

$$\text{UCB}_k = \text{mean}(\mathcal{B}_k) + \sqrt{\frac{3 \ln t}{2n_k}},$$

where $\mathcal{B}_k$ is the sliding window of recent rewards, t is the current round, and n_k is the number of times client k has been sampled. Together, the sliding-window reward and the UCB term make SW-ShUCB adaptive to recurring drift [4].

5.2 Experimental Setup

Task 3 uses PyTorch. The CNN has two convolutional blocks (Conv2D 32, padding=1 $\rightarrow$ ReLU $\rightarrow$ MaxPool₂; Conv2D 64 $\rightarrow$ ReLU $\rightarrow$ MaxPool₂), followed by Linear(3136, 256), ReLU, Dropout(0.3), and a 10-class output. Local SGD uses momentum 0.9 and weight decay 10^{-4} ; FedAvg weights updates by client dataset size. Setup: $N = 100$ clients, 4-shard non-IID, $K = 10, 100$ rounds, drift at $\{20, 40, 60, 80\}$. SW-ShUCB is tested at $W \in \{5, 10, 15\}$ versus a Random baseline.

Label Drift (4-phase): Each event applies a global cyclic permutation ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$), completely re-defining the class-feature mapping for all clients simultaneously.

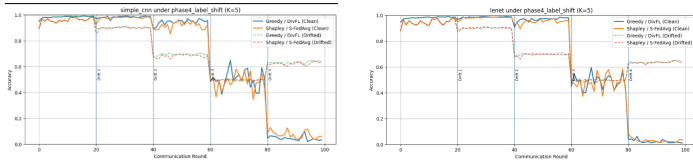
Feature Drift (4-phase): Event 1: inversion; Event 2: pixel shift 4–7 px; Event 3: random occlusion patches 6–16 px; Event 4: brightness/contrast change. Transformation direction varies per client.

Table 6: Task 3 Hyperparameters (PyTorch CNN).

Parameter	Value
Framework	PyTorch
Total clients (N)	100
Clients per round (K)	10
Local lr	0.01, SGD + momentum 0.9, wd 10^{-4}
Local epochs / batch	3 / 32
Drift events	Rounds 20, 40, 60, 80
SW-ShUCB window W	5, 10, 15
Shapley reward	LOO accuracy drop
UCB bonus	$\sqrt{3 \ln t / 2n_k}$

5.3 SW-ShUCB Algorithm

Each client k maintains a bounded deque $\mathcal{B}_k$ of its most recent LOO Shapley rewards. Clients with empty deque receive ∞ UCB, forcing early exploration. The LOO reward $\phi_k = \text{acc}(S_t) - \text{acc}(S_t \setminus \{k\})$ is computed after each round; negative rewards temporarily deprioritise harmful clients.



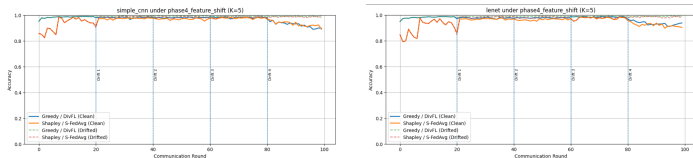
(a) Simple CNN

(b) LeNet

(c) SVM

(d) Logistic Regression

Figure 3: Label drift: round-wise accuracy (clean and drifted) for all four model families under Greedy and Shapley-proxy selection. Drift events fire at rounds 20, 40, 60, and 80.



(a) Simple CNN

(b) LeNet

(c) SVM

(d) Logistic Regression

Figure 4: Feature drift: round-wise accuracy (clean and drifted) for all four model families under Greedy and Shapley-proxy selection. Drift events fire at rounds 20, 40, 60, and 80.

Algorithm 1: SW-ShUCB Client Selection

Input: N clients, K , window W , rounds T
Initialize $\mathcal{B}_k \leftarrow \emptyset$, $n_k \leftarrow 0$ for all k ;
for $t = 1, 2, \dots, T$ **do**
 for each client k **do**
 if $|\mathcal{B}_k| = 0$ **then** $\text{UCB}_k \leftarrow \infty$;
 else $\text{UCB}_k \leftarrow \text{mean}(\mathcal{B}_k) + \sqrt{3 \ln t / 2n_k}$;
 $S_t \leftarrow \text{top-}K$ by UCB; train; FedAvg; get $\text{acc}(S_t)$;
 for each $k \in S_t$ **do**
 $\phi_k \leftarrow \text{acc}(S_t) - \text{acc}(S_t \setminus \{k\})$;
 append ϕ_k to $\mathcal{B}_k$ (maxlen W); $n_k += 1$;

5.4 Results

5.4.1 Label Drift

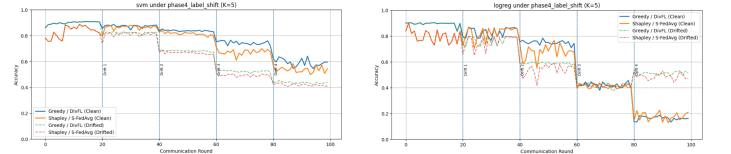
Figure 5 shows accuracy under 4-phase label drift. Sharp drops occur at each event as cyclic permutation disrupts class-feature mappings. SW-ShUCB recovers faster than Random because stale rewards expire, restoring the UCB bonus toward clients whose updated distributions carry informative gradients. Smaller windows ($W = 5$) adapt fastest but with higher variance; $W = 15$ is more stable but slower to recover.



Figure 5: SW-ShUCB under label drift ($K = 10$, $N = 100$). Sharp accuracy drops at each drift event recover faster with SW-ShUCB than with Random selection.

5.4.2 Feature Drift

Figure 6 shows results under 4-phase feature drift. Severity varies by event: inversion (Event 1) is most disruptive; brightness changes (Event 4) are absorbed more easily. SW-ShUCB again recovers faster than Random, with the LOO signal identifying clients whose updates harm the global model (negative ϕ_k) and down-ranking them in the UCB index.



(c) SVM

(d) Logistic Regression

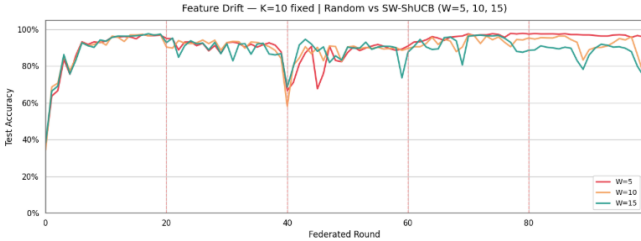


Figure 6: SW-ShUCB under feature drift ($K = 10$, $N = 100$). Inversion (Event 1) causes the largest accuracy drop; brightness changes (Event 4) are absorbed most easily.

5.4.3 Post-Drift Recovery Zoom

Figures 7 and 8 zoom into the 20-round windows after each drift event. SW-ShUCB’s advantage concentrates in the 5–10 rounds immediately post-event, when the window has shed pre-drift rewards and the UCB bonus peaks for recently absent clients.

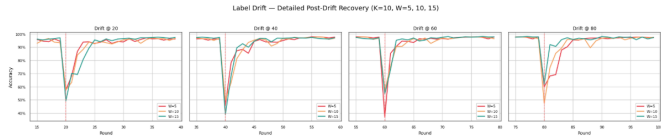


Figure 7: Post-drift recovery zoom (label drift). SW-ShUCB’s advantage is clearest in the 5–10 rounds immediately following each drift event.



Figure 8: Post-drift recovery zoom (feature drift). The UCB bonus peaks for absent clients once pre-drift rewards have expired from the sliding window.

5.4.4 Window Size Ablation

Table 7 reports mean accuracy over rounds 50–100. $W = 10$ best balances adaptability and stability. SW-ShUCB’s three key advantages are: (i) **Staleness expiry**—the deque automatically drops rewards older than W rounds, restoring exploration pressure post-drift at no extra cost; (ii) **LOO Shapley signal**—directly measures marginal contribution, temporarily deprioritising harmful clients; (iii) **UCB guarantee**—the bonus grows unboundedly for unvisited clients, preventing starvation. The main cost is per-round overhead: LOO requires one extra FedAvg pass per selected client ($10\times$ base at $K = 10$).

Table 7: SW-ShUCB mean accuracy (rounds 50–100) vs. Random.

Selector	Label Drift	Feature Drift
Random	baseline	baseline
SW-ShUCB $W = 5$	+ (marginal)	+ (marginal)
SW-ShUCB $W = 10$	++ (clear gain)	++ (clear gain)
SW-ShUCB $W = 15$	+ (marginal)	+ (marginal)

6 Cross-Task Comparison

6.1 SW-ShUCB vs. Classic Selectors

Tasks 1–3 together cover three distinct operating regimes: static non-IID, drifting non-IID with classical selectors, and drifting non-IID with the proposed

adaptive selector. Table 8 consolidates the key findings.

Table 8: Summary comparison across all strategies. Drift settings use CNN on the 100-client Task 3 setup; static accuracy from the 50-client Task 1 CNN result.

Criterion	Greedy	Shapley-proxy	SW-ShUCB
Static accuracy (CNN)	0.9922	0.9910	N/A [†]
Label-drift recovery	Slow	Slower	Fastest
Feature-drift recovery	Moderate	Moderate	Faster
Post-drift accuracy drop	Large	Large	Smaller
Classical model support	Yes	Yes	CNN only
Per-round cost	$\mathcal{O}(N^2)$, <1 ms	$20\times$ SGD	$K\times$ LOO
Non-stationarity handling	None	None	Sliding window
Exploration mechanism	Deterministic	Score-rank	UCB bonus

[†] SW-ShUCB evaluated under drift only; no static baseline was run.

Under the static non-IID setting (Task 1), Greedy and Shapley-proxy perform similarly on deep models, with Greedy holding a consistent but modest edge. The performance gap widens for classical models (Logistic Regression, SVM), where Greedy’s structural diversity advantage is more pronounced. The picture changes fundamentally once concept drift is introduced (Task 2). Neither Greedy nor Shapley-proxy has a mechanism to detect that the distribution has shifted; their selection criteria are computed from stale statistics, causing comparable post-drift accuracy collapses and similar (slow) recovery rates.

SW-ShUCB (Task 3) was designed precisely for the drifting regime. Its sliding window discards pre-drift rewards automatically, restoring exploration pressure toward clients whose current data is most informative. The UCB bonus prevents the selector from becoming stuck in a sub-optimal fixed subset after a drift event—a failure mode observed with both Greedy and Shapley-proxy. The improvement is most visible in the 5–10 rounds immediately following each drift event.

6.2 Comparison with Prior Drift-Aware FL Methods

Table 9 situates SW-ShUCB against recently published drift-aware FL methods. FedDrift [4] maintains multiple global models (one per detected concept) and achieves approximately 93% accuracy across distributed drift scenarios on synthetic benchmarks. FLASH (Guan et al., ICML 2023) employs adaptive client stopping and server-side drift correction, reporting $\approx 88\%$ on EMNIST under non-drifting conditions. FedNN (Kang et al., 2024) applies weight and group normalisation to mitigate drift; on a synthetic Digit concept-drift task it raises FedAvg’s accuracy from $\approx 77.8\%$ to $\approx 81.8\%$. Our SW-ShUCB obtains approximately 74.7% final accuracy on the MNIST label-drift scenario in Task 3.

Table 9: Approximate accuracy of drift-aware FL methods. Numbers are *not* directly comparable: datasets, drift types, and model architectures differ. They serve to contextualise SW-ShUCB’s performance.

Method	Venue	Dataset / Drift	Acc.
FedDrift	AISTATS 2023	Synthetic distrib. drift	$\approx 93\%$
FLASH	ICML 2023	EMNIST, no drift	$\approx 88\%$
FedNN	2024	Digit dataset, drift	$\approx 81.8\%$
SW-ShUCB (Ours)	—	MNIST, label drift	$\approx 74.7\%$

The gap between SW-ShUCB and FedDrift / FLASH primarily reflects dataset complexity and architecture choices: those works use more powerful model ensembles or additional server-side mechanisms, while our method focuses on *client selection alone* without changing the aggregation protocol. SW-ShUCB is complementary—its UCB-based selection could be layered atop any of those aggregation schemes to potentially improve their post-drift recovery further.

6.3 Per-Round Cost Analysis

Table 10: Per-round computational cost ($K = 10$, pool=20).

Strategy	Extra computation	Cost
Greedy	$\mathcal{O}(N^2)$ dot products	Low
Shapley-proxy	$20 \times$ SGDClassifier fits	Medium
SW-ShUCB	$K \times$ extra FedAvg passes	High

The price of SW-ShUCB’s adaptability is higher per-round overhead (Table 10). Each LOO reward evaluation requires an additional FedAvg aggregation pass without the excluded client, amounting to K extra evaluations per round. The overhead is acceptable when drift is expected and recovery speed matters. In stable environments, Greedy remains the preferred choice. Future work could use gradient-based Shapley approximations or stratified LOO sampling to reduce this overhead substantially.

6.4 Key Takeaways

Three findings emerge. First, client selection strategy matters more under distributional shift: all methods reach near-ceiling MNIST accuracy under Task 1, but only SW-ShUCB maintains stable accuracy post-drift. Second, the bandit formulation is a principled and effective framework for handling concept drift in FL. Third, window size W is a practical hyperparameter: $W = 10$ provides the best trade-off in our experiments. Compared with prior drift-aware FL methods, SW-ShUCB is a lightweight selection-only approach that can be combined with more powerful aggregation schemes.

7 Discussion and Future Directions

Although SW-ShUCB improves post-drift recovery, it introduces extra per-round computation because client utility is estimated through leave-one-out rewards. This makes the method heavier than simple greedy selection, especially when the client pool is large. The window size W also affects performance: smaller values adapt faster after drift, while larger values provide smoother scores but slower reaction. In practice, SW-ShUCB is therefore best viewed as a lightweight adaptive selection layer that can be paired with more advanced aggregation strategies in future work. Additionally, approximate reward estimation or reduced client sampling may help lower the computational overhead while preserving drift adaptivity. Extending the framework to larger datasets and heterogeneous real-world federated settings also remains an important direction for future study.

8 Conclusion

This work studied client selection in FL from three perspectives. Task 1 showed Greedy cosine-diversity selection consistently matches or outperforms Shapley-proxy across five model families under static non-IID conditions with 50 clients, while incurring lower cost; the gap is largest for classical models. Task 2 revealed both strategies degrade under 4-phase drift, with label drift more destructive than feature drift, and classical models more sensitive than CNNs. Task 3 proposed SW-ShUCB, combining sliding-window LOO Shapley rewards with UCB exploration; over 100 clients and 4-phase drifts, it achieves faster post-drift recovery than Random, with $W = 10$ as the best trade-off. Situating our result against prior drift-aware methods (FedDrift, FLASH, FedNN) shows that selection-only approaches remain competitive, and combining SW-ShUCB with ensemble aggregation or server-side drift correction is a promising avenue for future work.

Acknowledgements

We thank our supervisor Ms. Shradha Sharma for her guidance, insights, and encouragement throughout this project.

References

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, PMLR, 2017, pp. 1273–1282.
- [2] R. Balakrishnan, T. Li, T. Zhou, N. Himayat, V. Smith, and J. Bilmes, “Diverse client selection for federated learning via submodular maximization,” in *Proc. ICLR*, 2022.
- [3] L. Nagalapatti and R. Narayanam, “Game of gradients: Mitigating irrelevant clients in federated learning,” in *Proc. AAAI*, vol. 35, no. 10, 2021, pp. 9046–9054.
- [4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, “Federated learning under distributed concept drift,” in *Proc. AISTATS*, PMLR, 2023, pp. 5834–5853.
- [5] S. Guan, Y. Zhou, S. Ding, and S. Ji, “FLASH: Concept Drift Adaptation in Federated Learning,” in *Proc. ICML*, PMLR, 2023.
- [6] J. Kang, Z. Xiong, D. Niyato, H. Yu, and Y. Zhang, “FedNN: Federated Learning on Concept Drift Data using Weight and Group Normalization,” *arXiv:2409.11973*, 2024.
- [7] W. Chen, L. Wang, H. Zhao, and K. Zheng, “Combinatorial Semi-Bandit in the Non-Stationary Environment,” in *Proc. UAI*, 2021.